

# 位运算与 状压DP入门

主讲：黄凤林



## → 位运算

- ✧ 二进制操作又叫做位运算，这是计算机的基本，当然也是程序的基本，不过我们却很少去了解这个东西。
- ✧ 二进制有很多用处，常用来提高程序的效率，不过这些都是底层的東西，OI是用不上的，还有一个用处就是**压缩数据**，就是**用二进制来表示复杂的状态，然后通过位运算来完成问题的求解，这样能大大提高算法的效率，这样的状态表示经常用来优化DP或者是搜索。**
- ✧ 下面先讲讲常用的位运算

## ➔ 位运算---运算符

- ✧ **&** 按位与：如果两个相应的**二进制位都为1**，则该位的结果值为1，否则为0。
- ✧ **|** 按位或：两个相应的二进制位中**只要有一个为1**，该位的结果值为1。
- ✧ **^** 按位异或 若参加运算的两个**二进制位值相同则为0**，否则为1。
- ✧ **~** 取反 **~是一元运算符**，用来对一个二进制数按位取反，即将0变1，将1变0
- ✧ **<<** 左移 用来将一个数的各二进制位全部**左移N位**,右补0
- ✧ **>>** 右移 将一个数的各二进制位**右移N位**，移到右端的低位被舍弃，对于无符号数，高位补0



## → 位运算

### ✧ 左移

在很多语言里左移的符号位  $<<$ , 有些语言和伪代码常用 `shl` (pascal)

假设有个变量  $x=1$ ;

那么  $x=x<<1$  表示的意思如下:

$x$ : 000000000000000001 (二进制表示)

$x<<1$ : 000000000000000010 (也就是把  $x$  的每一位向左移动1位的结果, 右边不够的用0添上)

此时  $x=x<<1 == 2$ , 其实可以看做乘2

$x<<i$  表示的就是左移  $i$  位, 可以看做乘  $i$  次2



# → 位运算

## ✧ 右移

在很多语言里右移的符号位  $\gg$ , 有些语言和伪代码常用 `shr(pascal)`

假设有个变量  $x=2$ ;

那么  $x=x \gg 1$  表示的意思如下:

$x$ : 000000000000000010 (二进制表示)

$x \gg 1$ : 000000000000000001 (也就是把  $x$  的每一位向右移动1位的结果, 左边**不够的用0添上**)

此时  $x=x \gg 1 == 1$ , 其实可以**看做除2**

$x \gg i$  表示的就是右移  $i$  位, 可以看**做除  $i$  次2**



# → 位运算

## ✧ 或运算

在很多语言里或运算的符号位 **|**，这个与逻辑运算符 **||** 是有区别的，而一些语言和**伪代码也用 or** 表示

假如有  $x=(5)_{10}=(101)_2$ ,  $y=(2)_{10}=(010)_2$  (二进制表示)  
那么  $x|y$  表示如下

x: 101

y: 010

$x|y$ : 111

其实也就是**将x和y表示成二进制，然后按位做或运算**  
此时  $x|y=7$  (10进制表示)



# → 位运算

## ✧ 与运算

在很多语言里或运算的符号位 **&**，这个与逻辑运算符 **&&** 是有区别的，而一些语言和伪代码**也用 and 表示**

例如：x=101 y=010(二进制表示)

那么x&y表示如下

x: 101

y: 010

x&y: 000

其实也就是将x和y表示成二进制，然后按位做与运算  
此时x&y==0(10进制表示)



# → 位运算

## ✧ 非运算

在很多语言里或运算的符号位  $\sim$ ，而一些语言和伪代码也用 not 表示

例如：x=101(都是二进制表示)

那么 $\sim x$ 表示如下

x: 101

$\sim x$ : 010

其实也就是将x表示成二进制，然后按位做取反运算  
此时 $\sim x = 2$ (10进制表示)





## → 位运算

### ✧ 异或运算

在很多语言里或运算的符号位  $\wedge$ ，而一些语言和伪代码也用 **xor** 表示

例如：x=1010；y=1100 (二进制表示)

那么 $x \wedge y$ 表示如下

x: 1010

y: 1100

$x \wedge y$ : 0110

其实也就是将x和y表示成二进制，然后按位做异或运算，也就是**相同为0，不同为1**，此时 $x \wedge y = 6$ (10进制表示)



## → 位运算

### ✧ 获取一个或多个固定位的值 (&)

假设： $x=1100$ (10进制的10)

我们要获取从右往左数第3位的值，那么我们可以这样来取  
 $x \& (1 \ll 2)$ 也就是

$x : 1100$

$1 \ll 2 : 0100$

$x \& (1 \ll 2) : 0100$

这样我们就可以通过判断 $x \& (1 \ll 2)$ 是否等于0来知道这一位是0还是1了

当然我们可以用 $x \& (3 \ll 2)$ 来取得第3位和第4位

同时也可用 $x \& (x \ll 1)$ 来判断一个数相邻两位是不是同时为1



## → 位运算

### ✧ 把一个或多个固定位的值置零 (&)

假设:  $x = 1010$  (10进制的10)

我们把从右往左数第2位的值置零, 那么我们可以这样来做  
 $x \& (\sim(1 \ll 1))$  也就是

$x$ :                    1010

$\sim(1 \ll 1)$ :        1101

$x \& (\sim(1 \ll 1))$  1000

当然我们可以用  $x \& (\sim(3 \ll 2))$  来把第3位和第4位置零



## → 位运算

### ✧ 把一个或多个固定位的值取反 (^)

假设:  $x=1010$ (10进制的10)

我们把从右往左数第2位的值取反, 那么我们可以这样来做

$x \wedge (1 \ll 1)$  也就是; 如果  $x=1000$

$x$ : 1010      1000

$1 \ll 1$ : 0010      0010

$x \wedge (1 \ll 1)$ : 1000      1010

当然我们可以用  $x \wedge (3 \ll 2)$  来把第3位和第4位取反



## → 例题：牛吃草

题目描述：

给出一个 $n$ 行 $m$ 列的草地 ( $1 \leq M \leq 12; 1 \leq N \leq 12$ ), 1表示肥沃, 0表示贫瘠, 现在要把一些牛放在肥沃的草地上, 但是要求所有牛不能相邻, 问你有多少种放法 (mod 100000000)。

样例输入

2 3

1 1 1

0 1 0

样例输出

9

放牛的位置如下:

1 2 3

4

放0头牛, 有1个方案。

放1头牛(1、2、3、4), 有4个方案

放2头牛(13、14、34), 有3个方案

放3头牛(134), 有1个方案

$1 + 4 + 3 + 1 = 9$ 。



## ➔ 问题分析

- ✧ 如何解决这个问题呢?
- ✧ 思考：放牛的方式，即  
放1头牛的所有方案、放2头牛的所有方案、  
放3头牛的所有方案、.....  
放n头牛的所有方案，最后把结果加起来.....

**一行一行的来放牛**



## ➔ 问题分析

### ✧ 针对第1行放牛，则

- 1、放牛的方法最多是从12个0到12个1。
- 2、相邻两个位置不能都放牛（左右）。
- 3、满足题目中该位置是肥沃的，否则也是不能放牛的。

### ✧ 针对第2行放牛，则

- 1、放牛的方法最多是从12个0到12个1。
- 2、相邻两个位置不能都放牛（左右）。
- 3、还不能和第1行相同位置相邻（上线）
- 4、满足题目中该位置是肥沃的，否则也是不能放牛的。



## → 分析第 $i$ 行

✧ 针对第 $i$ 行放牛，则

- 1、放牛的方法最多是从12个0到12个1。
- 2、相邻两个位置不能都放牛（左右）。
- 3、还不能和第 $i-1$ 行相同位置相邻（上线）
- 4、满足题目中该位置是肥沃的，否则也是不能放牛的。



## ➔ 装压分析

- ✧ 由于每一行放牛中，如果我们按照每个位置存放起来，则比较相邻两格同时放牛是非常麻烦的。
- ✧ 因此我们应该考虑，把这行看成一个由0和1组成的二进制数来完成，从而实现状态的压缩。
- ✧ 这样思考后，我们按照行进行放牛，就发现满足DP的过程。
- ✧ 所以可以用DP来解决这个问题。
- ✧ 我们定义： $dp[i][j]$ :表示**第i行状态为j**的时候放牛的种数。
- ✧ **j的范围0~~12个1即  $(2^{12}-1)$ 。**
- ✧ 这样子就可以把一行的所有情况表示完成。



## → 分析3

- ✧ 状态压缩DP关键处理好位运算。这里处理的是：
- ✧ 1、一个二进制数相邻两位是不是同时为1（同行的判断）

**假如状态为j。  $j \& (j < < 1)$  来实现**

- ✧ 2、两个二进制数相同位是不是同时为1（上下行的判断）

**假如第i-1行状态为x,第i行状态为y。**

**$x \& y$  来判断相同位是否为1**



## ➔ 综合分析这个题目

✧ 针对每行DP，满足以下四点，则就是一种放置的方案。

- 1、放牛的方法最多是从12个0到12个1。
- 2、相邻两个位置不能都放牛（左右）。
- 3、还不能和第 $i-1$ 行相同位置相邻（上下）
- 4、满足题目中该位置是肥沃的，否则也是不能放牛的。

✧ 因此，针对第 $i$ 行状态为 $j$ 的方案数等于第 $i-1$ 行所有不与第 $i$ 行冲突的方案总数之和。



## ➔ 问题分析

- ✧ 分析第三点和第四点有什么区别
- ✧ 第三点中，通过上下相同位置是否存在一个都为1的便不能满足要求。所有可以直接 $x \& y$ ，如果返回值为0这个表示一个都不存在，否则就是存在
- ✧ 第四点中。要求当前第 $i$ 行的土地状态中某位是0的，则枚举的第 $i$ 行的状态的同位置必须是0，否则不满足要求（牛只能放在肥沃的土地上）。如何来寻找呢？我们发现找0是比较困难的。所以可以把原来土地中是0的变成1，是1的变成0（即取反）以后再转换成第三点的情况。

## → 问题分析

- ✧ 那处理土地按位取反的操作呢？有两种方式
- ✧ 第一次：读取的时候就直接取0的位

```
for(int i=1;i<=n;i++)  
{  
    for(int j=1;j<=m;j++)  
    {  
        scanf("%d",&x);  
        if(x==0) map[i]+=(1<<(m-j));  
    }  
}
```

- ✧ 第二种：正常读取，读完后在做&运算前，取个反  
if(x==1) map[i]+=(1<<(m-j));

```
bool judge2(int i,int j)//判断  
{  
    return ((~map[i])&st[j]);  
    //return (map[i]&st[j]);  
}
```



## ➔ 样例DP分析

✧ 土地状态

1 1 1

0 1 0

✧ 第1行状态有:

000、001、010、100、101 (思考为什么?)

对应:

$dp[1][0]=1$ 、 $dp[1][1]=1$ 、 $dp[1][2]=1$ 、 $dp[1][4]=1$ 、 $dp[1][5]=1$

✧ 第2行状态有:

000、010 (思考为什么?)

对应:  $dp[2][0]=5$ 、 $dp[2][2]=4$

✧ 所有合计有9种方案



# 实现代码：

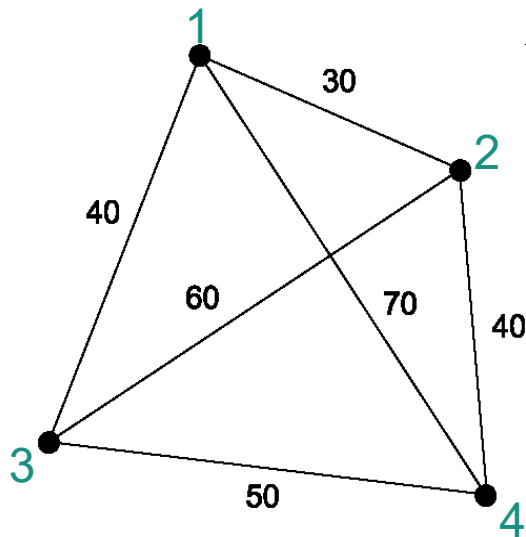
✧ 同学们可以自己完成了吗？



## 旅行商问题

### 问题描述：

旅行商要到若干个城市旅行，各城市之间的费用是已知的，为了节省费用，旅行商决定从所在城市出发，到每个城市旅行一次后返回初始城市，问他应选择什么样的路线才能使所走的总费用最短？ ( $N \leq 20$ )



从1出发  
最优路径 1 -> 2 -> 4 -> 3





# 问题分析

## ✧ 暴力搜索

始点固定，枚举所有路径。

复杂度  $O(n!)$  当 $n$ 大于10 ---TEL!!!

## ✧ 动态规划方法

状态定义：dp[i]表示：到达结点i点的最小花费。

正确么？

每个结点访问刚好一次---不满足无后效性

这是一道基于连通性状态压缩dp经典的题目

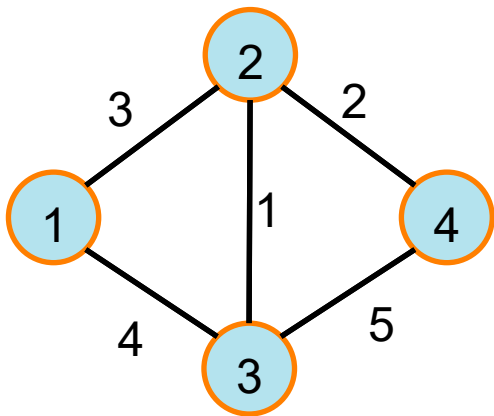


## → 问题分析

✧ 用 $dp[s][v]$ : 表示走完了点集 $S$ 后, 最后停留在 $v$ 点的最小花费。

✧ 递推方程:

$$dp[s][v] = \min(dp[(s \text{ 除去点 } v)][k] + \text{dis}[k][v], dp[s][v])$$



$$dp[1111][4] = \min(dp[1110][3] + \text{dis}[3][4], dp[1110][2] + \text{dis}[2][4])$$

$$dp[1111][2] = \min(dp[1011][3] + \text{dis}[3][2], dp[1011][4] + \text{dis}[4][2])$$

$$dp[1111][3] = \min(dp[1101][2] + \text{dis}[2][3], dp[1101][4] + \text{dis}[4][3])$$



## → 参考代码

```
for(int S = 0; S <= (1<<n)-1; ++S){ //枚举所有状态，用位运算表示
    for(int i = 1; i <= n; ++i)
    {
        if(S & (1<<(i-1)))
            //状态S中已经过城市i
        {
            if(S == (1<<(i-1))) dp[S][i] = dis[0][i];
            //状态S只经过城市I，最优解自然是从0出发到i的dis，这也是DP的边界
            else //如果S有经过多个城市
            {
                dp[S][i] = INF;
                for(int j = 1; j <= n; ++j)
                {
                    if(S & (1<<(j-1)) && j != i) //枚举不是城市I的其他城市
                        dp[S][i] = min(dp[S^(1<<(i-1))][j] + dis[j][i], dp[S][i]);
                    //在没经过城市i的状态中，寻找合适的中间点j使得距离更短，和FLOYD一样
                }
            }
        }
    }
}
```



## → noip2017 Day2 T2(宝藏)

- ✧ 题目大意:
- ✧ 给出一张无向图,每条边都有一个权值且均未开通,先可以随便取一个起点,要开通一些边,使它成为一个连通图,开通一条边的代价为权值\*起点到它的点的个数(起点也算),求最小代价。
- ✧ 数据范围:  $1 \leq n \leq 12$ ;  $0 \leq m \leq 1000$ ;  $v \leq 500000$ 。



## 宝藏—问题分析

- ✧ 问题分析：这个题和旅行商问题很相似，就是选取起点和路径不一样，所产生的代价就不一样， $n \leq 12$ ，所以处理方法可以状态压缩DP来实现。
- ✧  $x \& (x-1)$ 的作用：
- ✧ 将 $x$ 的二进制值中，最后一个1置0，其它不变



## 宝藏—问题分析

- ✧ 用 $f[S]$ 表示已经取的节点的状态为 $S$ 的最小代价，那么目标状态就是 $f[(1 \ll n) - 1]$
- ✧ 如果考虑递推求解的话，我们发现需要计算花费的深度会很难枚举，这就导致了DP顺序的问题，如果选择用记忆化搜索求解的话，会方便很多。
- ✧ 由于起点不是确定的，我们需要用一重循环来枚举起点，然后以 $1 \ll (\text{root} - 1)$ 为初始状态，对每一种情况进行一次记忆化搜索



# 记搜思路

- ✧ 记搜的大体思路如下：
- ✧ 1.对于状态 $S$ ，枚举一个点 $i$ ，满足 $i \in S$
- 2.再枚举一个点 $j$ ，满足 $i, j$ 有边相连且 $j \notin S$
- 3.状态转移方程： $f[S'] = \min(f[S] + \text{depth}[i] * \text{dis}[i][j])$
- 4.更新新加入的节点 $j$ 的深度
- 5.搜索下一个状态 $S'$
- 6.回溯还原节点 $j$ 的深度
- ✧ 最后在每一种情况的目标状态中取个 $\min$ 就是最终的答案了，时间复杂度 $O(n^3 2^n)$ 。

## 记忆化搜索参考代码

```
void dfs(int S)
{
    for(int i=1;i<=n;i++)
    {
        if((1<<(i-1))&S)
        {
            for(int j=1;j<=n;j++)
            {
                if(not ((1<<(j-1))&S)&&(dis[i][j]<INF))
                {
                    if(f[S|(1<<(j-1))]>f[S]+depth[i]*dis[i][j])
                    {
                        int temp=depth[j];
                        depth[j]=depth[i]+1;
                        f[S|(1<<(j-1))]=f[S]+depth[i]*dis[i][j];
                        dfs(S|(1<<(j-1)));
                        depth[j]=temp;
                    }
                }
            }
        }
    }
}
```



## ➔ 记忆化搜索参考代码

```
void solve()
{
    for(int root=1;root<=n;root++)
    {
        memset(f,0x3f,sizeof f);
        memset(depth,0x3f,sizeof depth);
        f[1<<(root-1)]=0;
        depth[root]=1;
        Search(1<<(root-1));
        ans=min(ans,f[(1<<n)-1]);
    }
}
```

## → 宝藏---问题分析

- ✧ 通过记忆化搜索发现，这里用一维 $dp[i]$ 表示时，不好枚举，也就是会导致有后效性，前面我说，当不行时，**那么我们为了使dp无后效性可以加一维甚至多维来限制，达到无后效性的效果。**考虑到这个题的特征，最终的答案是一棵生成树，所以我们**增加深度这一维来维护dp特征。**
- ✧ 用 $dp[i][s]$  表当前深度为 $i$ ,状态为 $j$ 的最小花费,则:
- ✧  $dp[i][s] = dp[i-1][t] + g[s][t] * (i-1)$ 。
- ✧  $g[s][t]$ 表示从**状态 $s$** 挖到**状态 $t$** 的最小花费。
- ✧ 时间复杂度 $O(3^n * n)$
- ✧ 为了预处理出 $g[t][s]$ ，我们可以再开一个 $h[i][s]$ 表示从点 $i$ 挖到状态 $s$ 的最小花费。
- ✧ 预算我们就可以用边权更新 $h$ 数组，用 $h$ 数组更新 $g$ 数组

## → 宝藏---问题分析

- ✧ h数组的预处理
- ✧  $h[i][s]$ 表示从点i挖到状态s的最小花费。

```
int ms=1<<n;
for (int i=1; i<=n; i++)
    for (int s=1; s<ms; s++)
    {
        h[i][s]=oo;
        if ( !(s&(1<<(i-1))) )
            for (int j=1; j<=n; j++)
                if ( s&(1<<(j-1)) )
                    h[i][s]=min(h[i][s],e[i][j]);
    }
```

## → 宝藏---问题分析

- ✧ g数组的预处理
- ✧  $g[i][j]$ 表示从i状态挖到j状态的最小花费

```
for (int s=1; s<ms; s++){  
    int t=s&(s-1);  
    while (t){  
        int x=s^t;  
        for (int i=1; i<=n; i++)  
            if ( x&(1<<(i-1)) )  
            {  
                g[s][t]+=h[i][t];  
                if (g[s][t]>oo) g[s][t]=oo;  
            }  
        t=s&(t-1);  
    }  
}
```



# 宝藏---Dp过程

```
for (int i=1; i<=n; i++)
    for (int s=0; s<ms; s++) f[i][s]=oo;
for (int i=1; i<=n; i++) f[1][1<<(i-1)]=0;
for (int i=2; i<=n; i++)
    for (int s=1; s<ms; s++)
    {
        int t=s&(s-1);
        while (t)
        {
            int temp;
            if (g[s][t]<oo) temp=g[s][t]*(i-1);
            else temp=oo;
            if (f[i-1][t]<oo) f[i][s]=min(f[i][s],f[i-1][t]+temp);
            t=s&(t-1);
        }
    }

int ans=oo;
for (int i=1; i<=n; i++) ans=min(ans,f[i][ms-1]);
printf("%d\n",ans);
```

题解博客: <https://blog.csdn.net/a1035719430/article/details/80488083>



## ➔ 状态压缩动态规划练习题

- ✧ <https://www.cnblogs.com/ibilllee/p/7651971.html>
- ✧ <https://www.cnblogs.com/Tony-Double-Sky/p/9283254.html>
- ✧ <https://www.cnblogs.com/ibilllee/p/7651971.html>
- ✧ 1、题目链接: <http://poj.org/problem?id=3254>
- ✧ 2、炮兵阵地—经典装压DP
- ✧ 3、[sgu225 装压DP&位运算](#)
- ✧ 状态压缩DP学习心得:
- ✧ <https://tbr-blog.blog.luogu.org/zhuang-ya-dp-chang-yong-cao-zuo>



## → 数位dp

- ✧ 数位统计类DP是与数字相关的一类计数问题。在这类题目中，一般给定一些限制条件，求满足**限制条件的第K小的数是多少，或者求在区间[L,R]内有多少个满足条件的限制**。解决方法一般先用动态规划进行预处理，再基于拼凑思想，用“试填法”求出最终答案。
- ✧ 数位dp的实质就是换一种暴力枚举的方式，使得新的枚举方式(按位枚举)满足dp的性质，然后记忆化就可以了。



## ➔ 数位dp

✧ 例：

求所有 $n$ 位数中能被 $m$ 整除的数的个数。

$N \leq 10^{15}$

问题分析：

朴素方法：从最小的 $n$ 位数枚举到最大的 $n$ 位数，然后判断是否能被 $m$ 整除即可。

时间复杂度 $O(n)$ ，**超时，不解释！** 怎么办？

寻求新的解决方法，转变思路，直接枚举数字大小是不行的，那我按照 $n$ 数字的位数，一位一位枚举，即数位DP。





# 问题分析

- ✧ 一般数位DP是多维的，首先会有一维表示的是当前到了第几位，通常情况这一维可以使用滚动。其它的就是根据题目的实际要求了，如例题就需要新增一维来记录除以 $m$  的余数。
- ✧ 以例题为例，我们来讲讲数位DP怎么实现。



## 状态与转移

- ✧ 设 $dp[i][j]$ 表示当前到了第 $i$ 位，除以 $m$ 的余数为 $j$ 的方案数。
- ✧ 一般是从高位往低位DP，因为最高位不能为0，所以我们对 $i=1$ 的特殊情况预先处理好： $dp[1][j \bmod m] = 1 (1 \leq j \leq 9)$
- ✧ 接着我们用第 $i$ 位的状态去更新第 $i+1$ 位的状态，每次枚举当前在第 $i+1$ 位加入 $k$ 。想想怎么转移？
- ✧ 对于当前的余数 $j$ ，在末尾加入了 $k$ ，余数变成什么？
- ✧ 根据同余，我们可以得到：余数变成了 $(j*10+k) \bmod m$ 。
- ✧ 所以，状态转移方程如下：
- ✧  $dp[i+1][(j*10+k) \bmod m] += dp[i][j] (1 \leq i < n, 0 \leq j < m, 0 \leq k \leq 9)$
- ✧ 最后的答案是什么？
- ✧ 首先，第一维自然是 $n$ ，
- ✧ 那么第二维取什么值呢？
- ✧ 显而易见，要求是 $m$ 的倍数，所以第二维是0。
- ✧ 综上所述，最后的答案是 $dp[n][0]$ 。



## → hdu:2089

- ✧ 题目大意:
- ✧ 从x到y求不包含62和4的数的个数。
- ✧  $0 < x \leq y < 1000000$
- ✧ 问题分析: 直接枚举, 肯定超时, 不解释
- ✧ 我们考虑求出0~y的个数, 再减去0~(x-1)的个数, 即为答案。
- ✧  $dp[i][j]$ : **表示i位数, 最高位是j的数, 符合题意的数有多少个。**
- ✧ 比如  $dp[1][2]=1$ ;  $dp[1][4]=0$ ;
- ✧  $dp[2][6]=8$  (60,61,63,64,65,66,67,68,69)
- ✧ 我们先不关注这个dp有什么用, 我们先关注dp本身怎么求。
- ✧ 首先  $dp[1][i]=0$  (if  $i=4$ ),  $dp[1][i]=1$  (if  $i \neq 4$ ) ( $0 \leq i \leq 9$ )。这一步是很显然的, 那么根据这个题的数据范围, 只需要递推到  $dp[7][i]$  就够用了。

## → 问题分析

✧ 那么稍微理解一下，可以想出递推式：

✧  $dp[i][j]=$

if ( $j==4$ )  $dp[i][j]=0$

else if ( $j!=6$ )  $dp[i][j]=\sum(dp[i-1][k])$  ( $k=0,1,2,3,4,5,6,7,8,9$ )

else if ( $j==6$ )  $dp[i][j]=\sum(dp[i-1][k])$  ( $k=0,1,3,4,5,6,7,8,9$ )

✧

```
void getdp()
{
    dp[0][0]=1;
    for(int i=1;i<10;i++){
        for(int j=0;j<10;j++){
            if(j==4) dp[i][j]=0;
            else if(j==6)
            {
                for(int k=0;k<10;k++) dp[i][j]+=dp[i-1][k];
                dp[i][j]-=dp[i-1][2];
            }
            else
            {
                for (int k=0;k<10;k++) dp[i][j]+=dp[i-1][k];
            }
        }
    }
}
```

✧

这样想：i位数，最高条件(因为题目不让便取，只要符合题如果j是6，后面只要和。

0的符合条件的数，



## ➔ 问题分析

- ✧ 那么现在我们已经得到了dp数组，再重申一下它的含义： $i$ 位数，最高位是 $j$ 的数，符合题意的数有多少个。
- ✧ 现在我们就关注怎么利用dp数组做出上面我们说的出 $[0, x]$ 中符合题意的数有多少个。
- ✧ 那么我们做这样一个函数`int solve(int x)` 它可以返回 $[0, x)$ 中符合题意的有多少个。
- ✧ 那么`solve(x+1)`实际上与 $[0, x]$ 是等价的。
- ✧ 那么现在问题转化成了：小于 $x$ ，符合题意的数多少个？
- ✧ 很简单，既然小于，从最高位开始比，必定有一位要严格小于 $x$ (前面的都相等)。所以我们就枚举哪一位严格小于(前面的都相等)。

## → 问题分析

- ✧ 假设我们现在把 $x$ 分成了 $a_1, a_2, \dots, a_L$ 这样一个数组，长度为 $L$ ， $a_L$ 是最高位。
- ✧ 那么结果实际上就是这样：
- ✧ 长度为 $L$ ，最高位取 $[0, a_L - 1]$ 的所有的符合题意数的和；再加上长度为 $L - 1$ ，最高位取 $a_L$ ，次高位取 $[0, a_{L-1} - 1]$ 的所有符合题意数的和；再加上……；一直到第一位。
- ✧ 我们还需要考虑这种情况：最高位 $a_L$ 如果是4，那么这句话直接就可以终止了，因为粗体这句话前面的那句话“最高位取 $a_L$ ”是不能成立的。还要考虑这种情况：最高位 $a_L$ 如果是6，那么这里并不是能取 $[0, a_{L-1} - 1]$ 的所有（不能取2）。加上这些条件之后就很严谨了。
- ✧ 上面的汉字对应到题目里，就是我们前面求出来的 $f[L][0..a_L - 1], f[L - 1][0..a_{L-1} - 1], \dots$



## → 练习题目

✧ hdu:3555:

给定x, 输出从0~x所有的数中, 含有49的所有数的个数。

✧ hdu:A serious math problem

给出一个整数x,求从0~x,可以被13整除, 并且包含13 的数的个数。

✧ hdu:F(X)

给定两个数a,b。还有一个加权函数,

$$F(x) = A_n \cdot 2^{n-1} + A_{n-1} \cdot 2^{n-2} + \dots + A_2 \cdot 2 + A_1 \cdot 1。$$

然后求, 0~b的各个数中进行F(x)不大于F(a)的数字的个数。



## ➔ 启示录(POJ 3208)

- ✧ 题目大意:
- ✧ 定义十进制下有3个连续的6的数为魔鬼数。有T个询问, 求第X小的魔鬼数。
- ✧  $X \leq 5 \times 10^7$ , 每个测试点包含不超过1000组数据。
- ✧ 问题分析:
- ✧ 1、朴素方法, 就是从最小的三位数开始枚举, 然后判断该数是否是魔鬼数。最后找到第x小的魔鬼数。时间复杂度为  $O(x \times 8^3 \times 1000)$ 。肯定是不行的。





## ➔ 启示录(POJ 3208)

- ✧ 用动态规划的思想来做，我们先确定第 $x$ 小的魔鬼数的位数，确定位数后，在用“试填”的方法，从左往右依次考虑每个数位。同时记录当前末尾已经有连续的几个6。
- ✧ 首先思考如何确定第 $x$ 小的魔鬼数的位数，直接做是没法做的，我们只能找出 $i$ 位数字有多少个魔鬼数来枚举处理。

## → 启示录(POJ 3208)

- ✧ 用 $F[i][3]$ 表示由 $i$ 位数字构成的魔鬼数有多少个。
- ✧  $F[i][j](0 \leq j \leq 2)$ 表示由 $i$ 位数字构成且开头已经连续有 $j$ 个6的非魔鬼数有多少个。
- ✧ 注意，在计算 $F$ 时，我们允许前导0的存在。
- ✧ 考虑第 $i$ 位(最高位)是什么数字，容易得到递归转移方程：
- ✧  $F[i][0] = 9 * (F[i-1][0] + F[i-1][1] + F[i-1][2])$
- ✧  $F[i][1] = F[i-1][0] \quad F[i][2] = F[i][1]$
- ✧  $F[i][3] = F[i-1][2] + 10 * F[i-1][3]$

```
void Init(){
    f[0][0]=1;
    for(int i=0;i<=20;i++){
        for(int j=0;j<3;j++){
            f[i+1][j+1]+=f[i][j];
            f[i+1][0]+=f[i][j]*9;
        }
        f[i+1][3]+=f[i][3]*10;
    }
}
```



## ➔ 启示录(POJ 3208)

- ✧ 通过 $F[i][3]$ 确定第 $X$ 小的魔鬼数的位数，然后在用“试填”的方法，从左往右依次考虑每个数位。同时记录当前末尾已经有连续的几个6。
- ✧ 从小到大枚举当前位数填什么数字，通过预处理的 $F$ 数组可以直接计算出后边的数位有多少种填法可以得到魔鬼数，与 $X$ 比较即可，代码实现很具技巧性。

## dp参考代码

```
while(f[m][3]<n) m++;
for(int i=m,k=0;i>0;i--){//试填第i位, 末尾已有K个6(k==3也表示已经是魔鬼数)
    for(int j=0;j<=9;j++){//从小到大枚举第i位填的数字
        long long cnt=f[i-1][3];//3个连续6全部来自于后i-1位
        if(j==6||k==3)//枚举的当前位是6或者已经有3个连续的6了
            for(int L=max(3-k-(j==6),0);L<3;L++)//妙用
                cnt+=f[i-1][L];
        if(cnt<n) n-=cnt;//如果cnt比n小, 说明第n个魔鬼数的第i位应该比j更大
        else//否则, 第i位就应该是j
        {
            if(k<3)
            {
                if(j==6) k++; else k=0;
            }
            printf("%d",j);
            break;
        }
    }
}
```



## → 月之谜(BZOJ 1799)

- ✧ 题目大意:
- ✧ 如果一个十进制数能够被它的各位数字之和整除, 则称这个数为“月之数”。给定整数L和R, 你需要计算闭区间[L, R]中有多少个月之数。
- ✧  $1 \leq L, R \leq 2^{31}$ , 每个测试点包含不超过3000组数据。

The background features a central point from which several wide, colorful rays (in shades of blue, orange, red, and grey) radiate outwards. Faint binary code (0s and 1s) is scattered across the background, and a line graph with two fluctuating lines (one blue, one orange) is visible on the right side.

# Thank you!